

From Promoter Model Training to Annotated SBOL Designs

A concise teaching deck for using SeqTrainer annotation features, exporting GenBank/SBOL outputs, validating design files, and visualizing results in SBOL Canvas or GenBank viewers.

Training notebooks

Promoter annotation

SBOL export

Canvas visualization

Learning outcomes

By the end of this guide, students should know what each SeqTrainer artifact is for.

1

Set up SeqTrainer from the official repository

Clone SynBioDex/SeqTrainer, install extras, and work from the main branch used by the university lab.

2

Retrain or rerun notebooks responsibly

Edit hyperparameters while preserving validation/test separation and benchmark provenance.

3

Run promoter annotation

Use a GenBank file, checkpoint, and benchmark manifest to produce computational predicted promoters.

4

Export and inspect results

Read annotated GenBank, predictions CSV, annotation manifest, SBOL3, and SBOL2/RDF outputs.

5

Validate and visualize

Use SBOL Validator and SBOL Canvas for standards checks and visual inspection.

Official setup: start from SynBioDex main

Use this as the first practical cell or terminal block in a university lab handout.

```
# 1) Clone the official repository
git clone https://github.com/SynBioDex/SeqTrainer.git
cd SeqTrainer

# 2) Work from the course-approved main branch
git checkout main
git pull origin main

# 3) Create an isolated Python environment
python -m venv .venv
source .venv/bin/activate    # Windows: .venv\Scripts\activate

# 4) Install SeqTrainer locally
python -m pip install --upgrade pip
python -m pip install -e "[annotation,dev]"
```

Teaching instruction

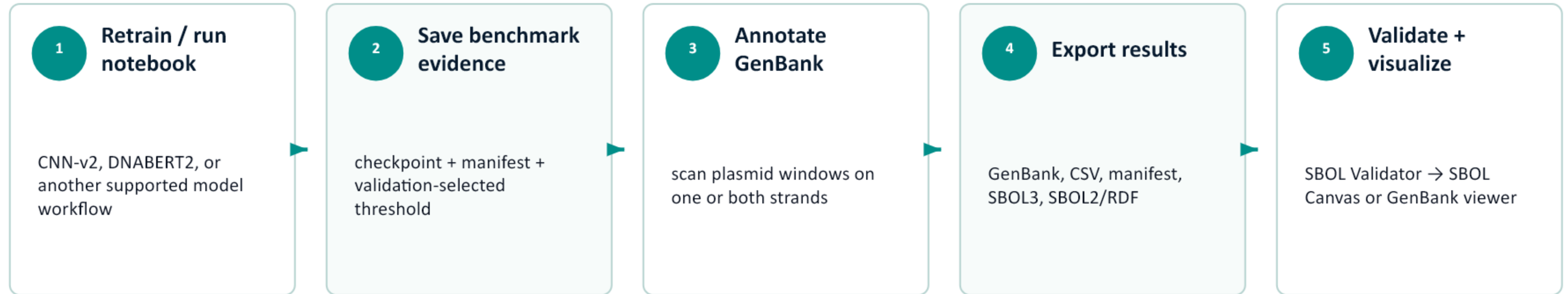
Ask students to confirm that `seqtrainer --help` works before running any annotation command. This catches most path and environment errors early.

If annotation is not merged yet

Use the exact branch specified by the instructor or PR. The slide commands should still be taught as the intended SynBioDex main workflow after merge.

Overall workflow: one biological question

How do we move from a promoter model to predicted promoter features in a plasmid?



Central rule

Do not treat every high-scoring window as a confirmed promoter. SeqTrainer adds computational predicted promoter features that require biological interpretation and, ideally, experimental validation.

Part A : Retrain or rerun a model notebook

Training creates the checkpoint and manifest needed later for annotation.

```
# From the official SeqTrainer checkout
cd SeqTrainer

# Start notebooks
jupyter lab

# Open the course notebook under notebooks/
# Example naming pattern:
# notebooks/.../cnn_v2_*.ipynb
# notebooks/.../dnabert2_*.ipynb
```

Notebook teaching flow

1. Load the shared dataset
2. Preprocess sequences
3. Train or load the model
4. Evaluate on validation
5. Select threshold from validation
6. Report held-out test metrics
7. Save checkpoint and manifest

Annotation should begin only after a model run has a checkpoint and a manifest that records the training/evaluation settings.

What can students safely edit in notebooks?

Use validation performance to guide training changes, not repeated test-set trial-and-error.

1

Learning rate

Controls how strongly model weights update. Too high can destabilize training; too low may learn slowly.

2

Epochs

Number of full passes through the training data. More epochs can overfit.

3

Batch size

Affects memory use and optimization noise. Large models may require smaller batches.

4

Dropout / regularization

Reduces overfitting by discouraging fragile learned patterns.

5

Threshold policy

Should be selected on validation metrics such as MCC, then frozen for test and annotation.

6

Random seed

Record it so another student can reproduce the run.

Benchmark outputs that annotation needs

A training notebook is useful only if it leaves auditable artifacts.

checkpoint

model weights used for inference

manifest.json

model family, threshold, dataset and run metadata

metrics.csv/json

validation and test performance summary

predictions.csv

row-level probabilities and predicted labels

history.csv

training curve if model was trained

The benchmark manifest connects model training to annotation. It prevents the annotation step from becoming a disconnected one-off prediction script.

Part B : Prepare the annotation input

The annotation command starts from an existing GenBank DNA record.

Input file

A GenBank file such as `pAN1717\_cyan.gb` or any course-provided plasmid record. Existing features should be preserved rather than overwritten.

Model evidence

A checkpoint and benchmark manifest from a completed model run. These define the predictor and threshold provenance.

Output folder

Use a clean folder such as `outputs/annotations/` so the GenBank, CSV, manifest and SBOL files stay together.

```
mkdir -p outputs/annotations

# Recommended project structure
inputs/
  plasmid.gb
outputs/
  benchmarks/
    model_run/manifest.json
    model_run/checkpoints/best_model.pt
  annotations/
```

Annotation smoke test: confirm the pipeline works

The dummy predictor is for file writing and workflow checks only. It is not a biological model.

```
seqtrainer annotate promoters inputs/plasmid.gb \  
  --model-family dummy \  
  --threshold 0.80 \  
  --window-size 300 \  
  --step-size 25 \  
  --scan-both-strands \  
  --output outputs/annotations/plasmid_dummy_annotated.gb \  
  --predictions-csv outputs/annotations/plasmid_dummy_predictions.csv \  
  --manifest outputs/annotations/plasmid_dummy_manifest.json \  
  --clean-output
```

Use this first

Run this before using a real checkpoint. It checks that SeqTrainer can read the GenBank file, generate windows, write features, export the CSV, and record a manifest.

Do not report dummy results

The dummy predictor is not trained. It should never be used as biological evidence.

Real annotation command: checkpoint + manifest

Replace the paths with the model run produced by your course notebook.

```
seqtrainer annotate promoters inputs/plasmid.gb \  
  --model-family dnabert2 \  
  --checkpoint outputs/benchmarks/dnabert2_run/checkpoints/best_model.pt \  
  --benchmark-manifest outputs/benchmarks/dnabert2_run/manifest.json \  
  --output outputs/annotations/plasmid_dnabert2_annotated.gb \  
  --predictions-csv outputs/annotations/plasmid_dnabert2_predictions.csv \  
  --manifest outputs/annotations/plasmid_dnabert2_manifest.json \  
  --sbol-output outputs/annotations/plasmid_dnabert2.nt \  
  --sbol2-output outputs/annotations/plasmid_dnabert2_sbol2.rdf
```

Model family choices

`dnabert2` and `cnn\_v2` require compatible checkpoints. Keep model family, checkpoint and manifest from the same benchmark run.

SBOL outputs

Use SBOL3 `.nt` as the standards-oriented machine-readable export. Use SBOL2/RDF `.rdf` for SBOL Canvas compatibility when required.

Editable annotation parameters

These settings change how SeqTrainer scans the input DNA.

1 --window-size

Length of sequence shown to the model at once. Keep compatible with training length.

2 --step-size

Distance between neighboring windows. Smaller step = denser scan and more runtime.

3 --threshold

Probability cutoff. Prefer the validation-selected value stored in the benchmark manifest.

4 --scan-both-strands

Scores forward and reverse-complement orientation. Useful for plasmids with features on both strands.

5 --clean-output

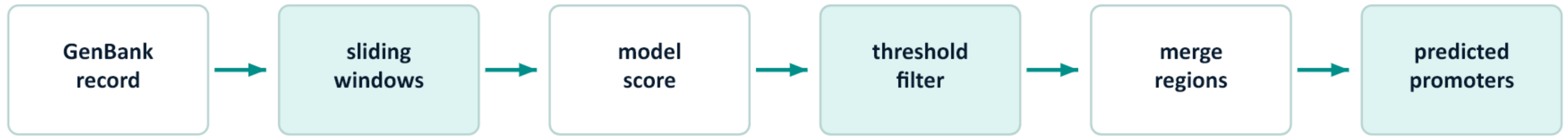
Removes the target output files before a fresh run. Avoids confusing stale files.

6 --open-output-folder

Convenience option for local teaching demos when supported.

What SeqTrainer does internally

This is the scientific core of the annotation workflow.



```
window_start, window_end, strand, score, threshold_pass,  
merged_region_id, overlaps_existing_feature, circular_boundary
```

Every scanned window is saved in the predictions CSV, so students can inspect both positive and negative windows instead of relying only on the final GenBank file.

Output 1 : Annotated GenBank

Use this for sequence-level inspection in GenBank-compatible viewers.

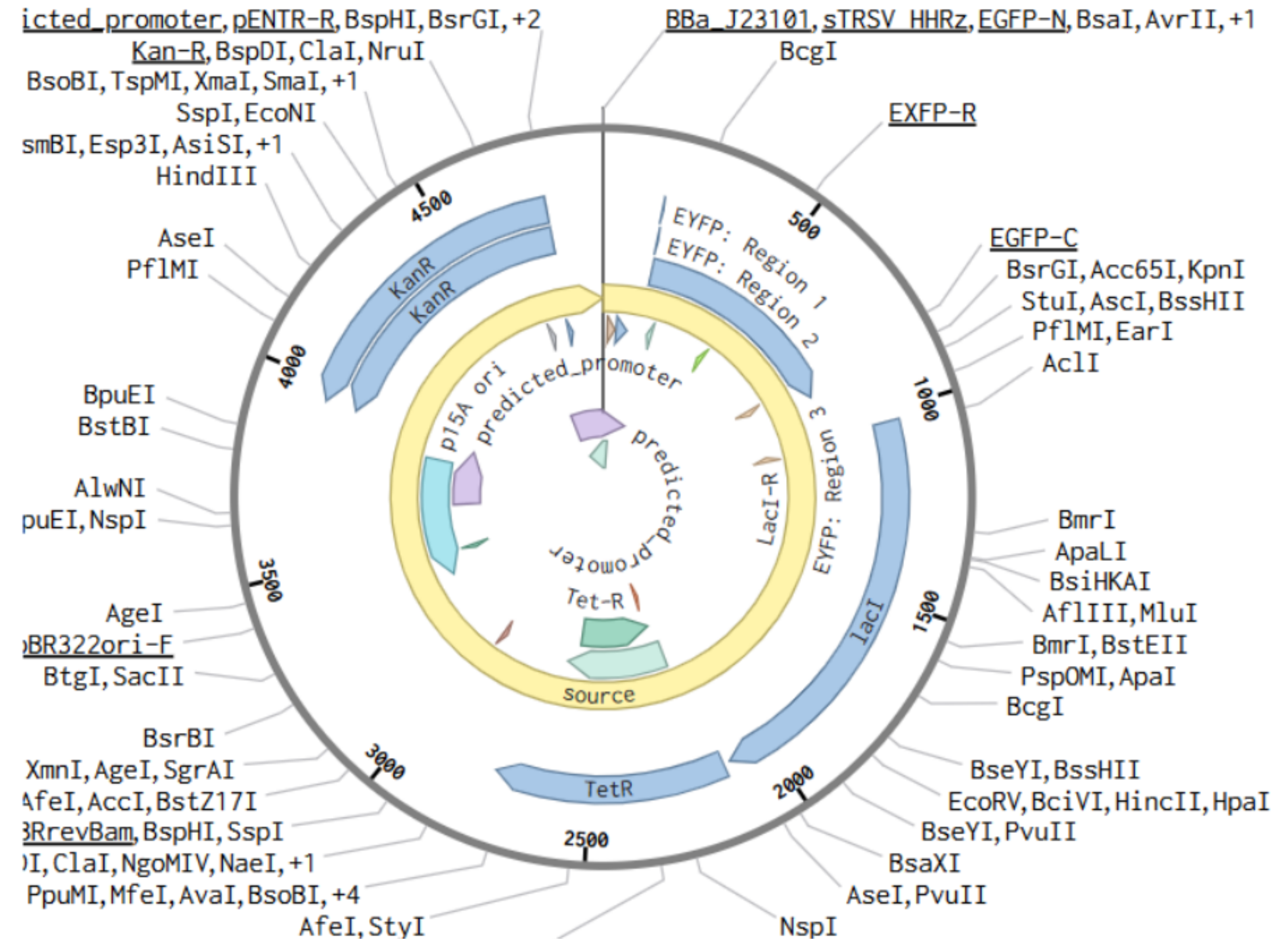
What changes?

Existing features remain in the record. SeqTrainer appends additional computational features labelled as predicted promoters.

What not to claim

A predicted promoter is not a confirmed biological part. Treat it as a model-derived candidate region.

```
# Open in Benchling, SnapGene, ApE,  
Geneious, or another GenBank viewer  
outputs/annotations/plasmid_dnabert2_anno  
tated.gb
```



Example visual inspection of predicted_promoter features in a sequence viewer / SBOL Canvas-style map.

Output 2 : Predictions CSV and annotation manifest

These are the files that make the annotation auditable.

predictions.csv

One row per scanned window. Use it to inspect coordinates, strand, score, threshold result, overlap status, region ID, and sequence.

annotation_manifest.json

Records input/output paths, model family, checkpoint, benchmark manifest, threshold source, window settings, circular topology, warnings and timestamp.

```
python - <<'PY'  
import pandas as pd  
df =  
pd.read_csv('outputs/annotations/plasmid_dnabert2_pre  
dictions.csv')  
print(df.sort_values('score',  
ascending=False).head(10))  
'PY
```

```
python -m json.tool \  
  
outputs/annotations/plasmid_dnabert2_manifest.json  
| head -80
```

Output 3 : SBOL export

Use SBOL when the design needs to move into standards-oriented synthetic-biology tools.

SBOL3 .nt

Machine-readable representation of the sequence, mapped features, predicted promoter features and inference provenance. Use it for standards-oriented exchange and archiving.

SBOL2 .rdf

Compatibility export for tools that expect SBOL2 RDF/XML, especially SBOL Canvas workflows that do not directly consume the SBOL3 N-Triples file.

```
# Typical SeqTrainer annotation command already creates both:  
--sbol-output outputs/annotations/plasmid_dnabert2.nt \  
--sbol2-output outputs/annotations/plasmid_dnabert2_sbol2.rdf
```

SBOL validation checks file structure and standards compliance. It does not prove that the predicted promoter is biologically active.

Validate the SBOL file when required

Use the official SBOL Validator before opening or sharing designs.

Web workflow

1. Open validator.sbolstandard.org
2. Upload the SBOL output file
3. Select the relevant SBOL version / conversion option
4. Save validation messages and converted files with your run outputs

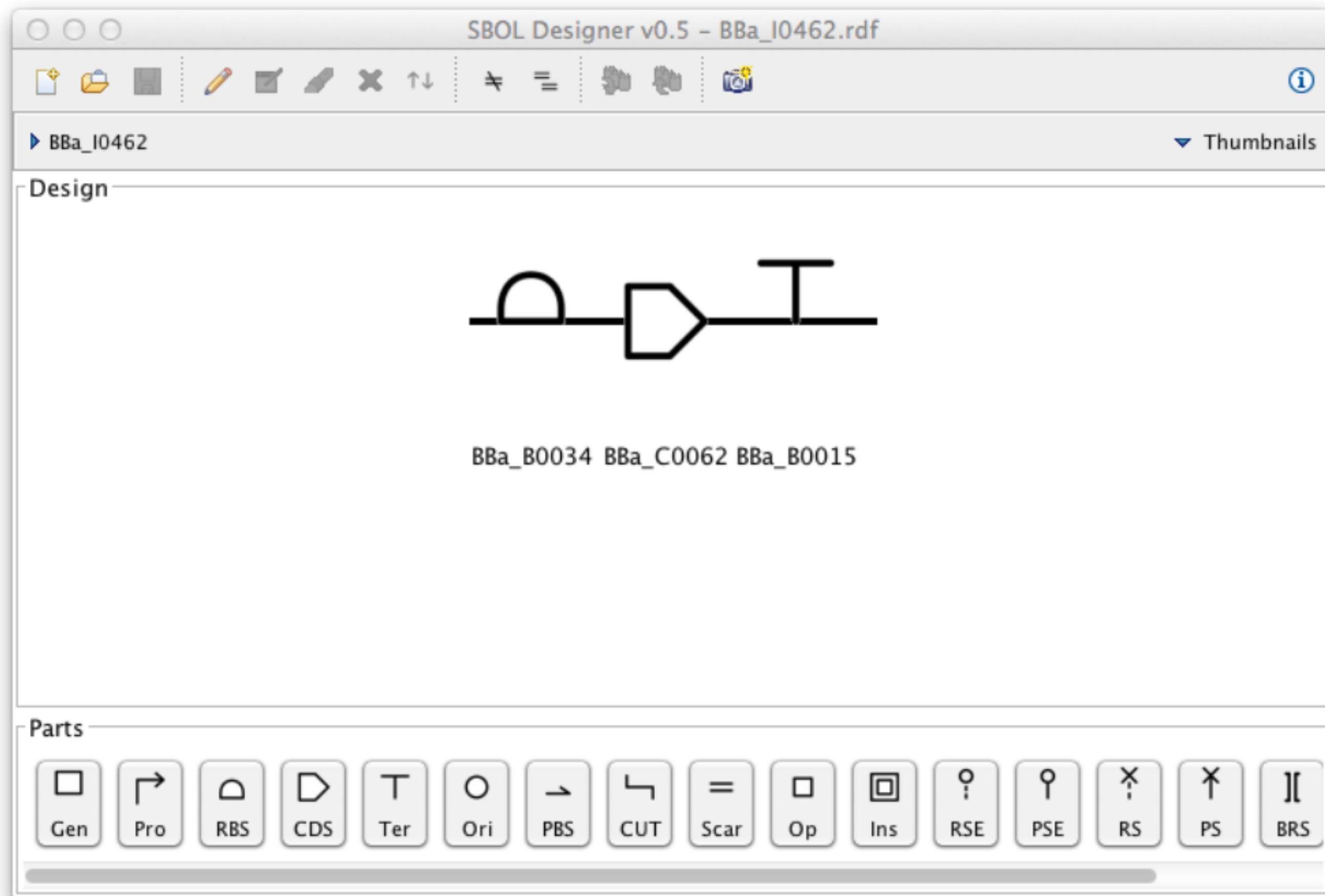
What to save

- SBOL3 .nt
- SBOL2 .rdf compatibility file
- validation report or messages
- annotation manifest
- original GenBank input

```
outputs/annotations/  
  plasmid_dnabert2_annotated.gb  
  plasmid_dnabert2_predictions.csv  
  plasmid_dnabert2_manifest.json  
  plasmid_dnabert2.nt  
  plasmid_dnabert2_sbol2.rdf  
  sbol_validation_report.txt
```

Visualize in SBOL Canvas

Use the Canvas-compatible SBOL2/RDF output for visual inspection.



Canvas handoff

1. Open sbolcanvas.org
2. File → Import
3. Upload `plasmid\_dnabert2\_sbol2.rdf`
4. Inspect predicted_promoter features
5. Save/export visual design if needed

Important

Open the SBOL2 `.rdf` compatibility export in Canvas. Keep the SBOL3 `.nt` file as the canonical standards-oriented machine-readable output unless the tool explicitly supports it.

Use Seq Design Buddy as a workflow signpost

The website should explain the handoff; it should not run the model or store files.

What it should do for students

- Explain which SeqTrainer command to run
- Distinguish GenBank, CSV, manifest, SBOL3 and SBOL2 outputs
- Link to official SeqTrainer, SBOL Validator and SBOL Canvas pages
- Provide copyable commands and glossary terms

What it should not do

- Upload biological files
- Run DNABERT2/CNN jobs
- Claim validation results
- Store model outputs
- Replace official SeqTrainer documentation

Recommended route label

SeqTrainer annotation → SBOL validation → SBOL Canvas visualization → optional GenBank viewer inspection

Academic checklist before reporting results

- 1 Repository is SynBioDex/SeqTrainer, main branch or course-approved branch.
- 2 Notebook changes are recorded: seed, learning rate, epochs, model family and split paths.
- 3 Threshold is selected from validation, not the held-out test set.
- 4 Checkpoint and benchmark manifest are from the same run.
- 5 Annotation output distinguishes predicted_promoter from confirmed biological parts.
- 6 Predictions CSV and manifest are submitted with the GenBank/SBOL files.
- 7 SBOL Validator messages are saved when SBOL is used.
- 8 SBOL Canvas visualization is interpreted as visual inspection, not biological validation.

Suggested university practical session

A 2-hour teaching plan.

1 0–20 min: Setup

Clone SynBioDex/SeqTrainer, install extras, run ``seqtrainer --help``.

2 20–45 min: Notebook review

Open the course notebook and identify hyperparameters, metrics and output artifacts.

3 45–75 min: Annotation

Run dummy annotation, then real model annotation if a checkpoint is supplied.

4 75–100 min: Inspect outputs

Open GenBank, predictions CSV and manifest. Explain score, threshold and strand.

5 100–115 min: SBOL

Export/validate SBOL files and explain SBOL3 vs SBOL2/RDF roles.

6 115–120 min: Discussion

Why prediction is not biological validation; what wet-lab follow-up would be needed?

Source notes for instructors

SeqTrainer repository	https://github.com/SynBioDex/SeqTrainer
SBOL standard website	https://sbolstandard.org/
SBOL Visual glyphs	https://sbolstandard.org/visual-glyphs/
SBOL Validator	https://validator.sbolstandard.org/
SBOL Canvas	https://sbolcanvas.org/
SBOL Canvas repository	https://github.com/SynBioDex/SBOLCanvas

Thank you

Questions and discussion

Suggested discussion prompt: How would you experimentally validate a computationally predicted promoter region?

Key message

SeqTrainer annotations are traceable computational hypotheses. SBOL and Canvas help exchange and inspect them; experiments validate them.